T-GRINS

TLUSTY Grid Interface for NLTE Synthesis

User Manual — Version 1.0

Developed by

Felipe I. Figueroa-Tapia

Instituto de Física y Astronomía
Universidad de Valparaíso, Chile

felipe.figueroat@postgrado.uv.cl
felipe.figueroat@proton.me

https://felipefigueroat.github.io/
https://massivestars.ifa.uv.cl/

Abstract. **T-GRINS** is a pipeline for computing grids of non-LTE (NLTE) model atmospheres of OB stars using TLUSTY 208 (Hubeny & Lanz, 2017a) and SYNSPEC 54 (Hubeny et al., 2021), submitted in parallel via HTCondor. The pipeline covers the full parameter space relevant to massive OB stars ($T_{\rm eff}$ = 25 000–50 000 K, $\log g$ = 2.5–4.3) with explicit treatment of H, He, C, N, O, Si, S, and Fe up to Fe VI (O stars) or Fe III (B stars). Each converged model produces a compressed JLD2 file containing the detailed spectral energy distribution (SED) from 50 to 50 000 Å, suitable for use in stellar population synthesis, mass-loss rate determinations, and photometric modeling. This manual describes the physical setup, the pipeline architecture, all configuration options, and the output file format.

April 24, 2026

Contents

1 Introduction **3**
1.1 Pipeline Overview . . . 3

2 Requirements and Installation **4**
2.1 TLUSTY and SYNSPEC . . . 4
2.1.1 Recompiling SYNSPEC . . . 4
2.2 Julia Packages . . . 5
2.3 Python Packages (optional) . . . 5

3 Repository Structure **5**

4 Quick Start **6**
4.1 Setting Up a New Grid Directory . . . 6
4.2 Running a Single Model . . . 6
4.3 Monitoring and Collecting Results . . . 6

5 Input File Format **7**
5.1 Grid Mode Input . . . 7
5.2 Config Mode Input . . . 7

6 Physical Setup **8**
6.1 NLTE Pipeline Steps . . . 8
6.2 Elements Included . . . 8
6.3 Eddington Parameter and Convergence Limits . . . 9
6.4 Metallicity Scaling . . . 9
6.5 SYNSPEC Wavelength Coverage . . . 9

7 Configuration Parameters **10**
7.1 TLUSTY Parameters . . . 10
7.2 SYNSPEC Parameters . . . 10
7.3 Output Parameters . . . 10

8 Output Format **11**
8.1 JLD2 File Structure . . . 11
8.2 Reading JLD2 in Python . . . 11
8.3 Reading JLD2 in Julia . . . 12
8.4 Directory Contents After a Successful Run . . . 12

9 Retry Strategy for Difficult Models **12**
9.1 Motivation . . . 12
9.2 Generating the Retry List . . . 12
9.3 Running the Retry . . . 13

10 HTCondor Integration **13**
10.1 Submit File Structure . . . 13
10.2 Memory Requirements . . . 14

11 Plotting **14**

12 Troubleshooting **14**

13 Quick Reference **15**

1 Introduction

Accurate stellar atmosphere models are a cornerstone of hot-star astrophysics. They provide the emergent radiation field needed to determine stellar parameters from observed spectra, to model ionising photon budgets in H II regions, and to compute mass-loss rates from UV P-Cygni profiles. The two most widely used publicly available NLTE grids for OB stars are OSTAR2002 (Lanz & Hubeny, 2003) and BSTAR2006 (Lanz & Hubeny, 2007), both computed with TLUSTY and covering solar metallicity and several sub-solar compositions.

T-GRINS extends this effort by providing a modern pipeline built on TLUSTY 208 and SYNSPEC 54 (Hubeny et al., 2021), which include several improvements over the versions used in OSTAR2002/BSTAR2006: a more efficient ODF treatment for iron line blanketing via `ISPODF`, and an updated atomic data set. The pipeline is designed to be reproducible, fully configurable, and straightforward to deploy on any HTCondor cluster.

The main capabilities of T-GRINS are:

- Automatic generation of all TLUSTY and SYNSPEC input files from a simple text table of $(T_{\rm eff}, \log g)$ pairs.
- A four-step NLTE convergence strategy that introduces Fe blanketing gradually, avoiding the divergence problems common in near-Eddington models.
- Multi-segment SYNSPEC execution that circumvents the internal Fortran array overflow in the dense far-UV line list.
- Metallicity scaling relative to Asplund et al. (2009) solar abundances, including a global $Z/Z_\odot$ parameter and per-element overrides.
- A retry pipeline using converged neighbor models as starting points for models that fail to converge from a gray LTE atmosphere.
- Compressed JLD2 output files readable with `h5py` (Python) or `JLD2.jl` (Julia).

1.1 Pipeline Overview

Figure 1 illustrates the five-step pipeline executed for each model.

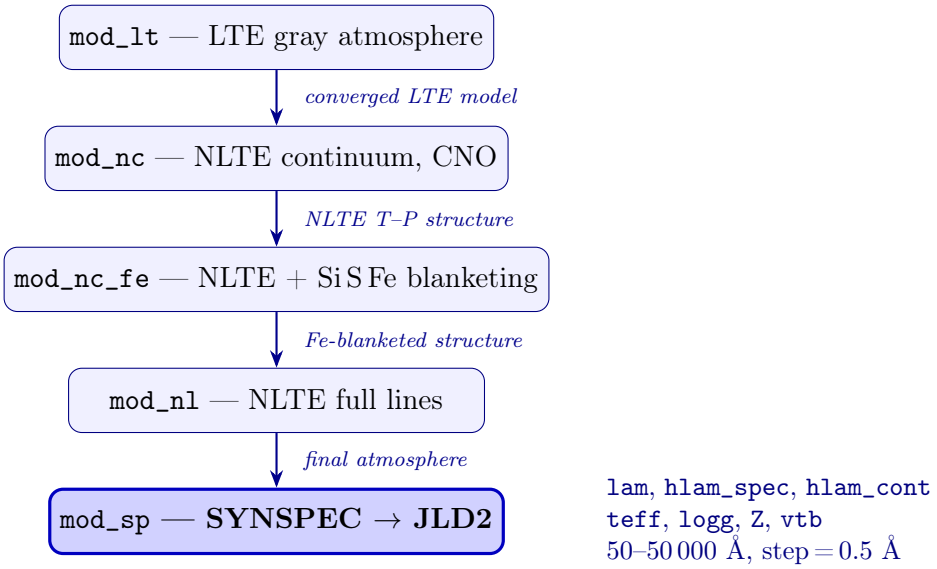

Figure 1: The five-step NLTE pipeline executed by `run_model_fe.jl`. Each step uses the converged model of the previous step as its starting point. SYNSPEC is run in two segments for O stars to handle the dense far-UV iron line list.

2 Requirements and Installation

2.1 TLUSTY and SYNSPEC

T-GRINS requires TLUSTY 208 and SYNSPEC 54, freely available from http://tlusty.oca.eu/. After downloading and unpacking, set the following environment variables (e.g., in `~/.bashrc` or in the Condor environment):

```
export TLUSTY=/path/to/tl208-s54
export LINELIST=/path/to/linelist  # directory with gfATO.dat or gfATO.bin
export IRON=/path/to/irondata      # Kurucz iron superline tables
export OPTABLES=/path/to/optables
```

2.1.1 Recompiling SYNSPEC

The standard SYNSPEC binary is compiled with `MLINO = 1 200 000` internal line-list entries, which is insufficient for the dense far-UV region of O star spectra (the `gfATO` line list contains ∼1.48 million lines below 900 Å). Before running T-GRINS, increase the array sizes in `$TLUSTY/synspec/LINDAT.FOR`:

```
# Edit LINDAT.FOR
sed -i 's/MLINO =1200000/MLINO =3000000/' $TLUSTY/synspec/LINDAT.FOR
sed -i 's/MLIN  = 190000/MLIN  = 500000/' $TLUSTY/synspec/LINDAT.FOR

# Recompile
cd $TLUSTY/synspec
gfortran -fno-automatic -mcmodel=large -O3 -o synspec.exe synspec54.f
```

Do not skip the recompilation step. The default SYNSPEC binary will silently stop after processing the first spectral window when run on O stars with wavelength coverage starting below 900 Å, producing a degenerate output of only 235 points.

2.2 Julia Packages

```
julia -e 'using Pkg; Pkg.add(["JLD2", "CodecZlib", "TOML"])'
# Optional (for plot/plot_sed.jl):
julia -e 'using Pkg; Pkg.add("Plots")'
```

Alternatively, install all dependencies from `Project.toml` at the repository root:

```
julia --project=. -e 'using Pkg; Pkg.instantiate()'
```

2.3 Python Packages (optional)

Only required for `plot/plot_sed.py` and `examples/read_jld2.py`:

```
pip install h5py numpy matplotlib
```

3 Repository Structure

```
T-GRINS/
|-- README.md
|-- Project.toml        Julia dependencies
|-- config.toml         Template for single-model mode
|
|-- pipeline/
|   |-- run_model_fe.jl Main pipeline (grid or single mode)
|   |-- run_wrapper.sh  HTCondor wrapper
|   |-- tlusty_grid.sub HTCondor submit file
|   |-- run_retry.jl    Retry pipeline for difficult models
|   |-- retry_wrapper.sh HTCondor wrapper for retry
|   \-- retry_grid.sub  HTCondor submit file for retry
|
|-- tools/
|   |-- setup_grid.sh   Create a ready-to-run grid directory
|   |-- make_retry_list.jl Generate retry pairs (model, neighbor)
|   |-- check_grid.jl   Report grid status
|   |-- check_retry.jl  Report retry status
|   |-- collect_jld2.jl Collect JLD2 files into a single directory
|   \-- make_jld2.jl    Regenerate JLD2 from existing ASCII output
|
|-- plot/
|   |-- plot_sed.jl     Plot SED from JLD2 (Julia)
|   \-- plot_sed.py     Plot SED from JLD2 (Python)
|
\-- examples/
    |-- tlusty-input_example.dat Sample input file (9 models)
    \-- read_jld2.py    Reading JLD2 files in Python
```

4 Quick Start

4.1 Setting Up a New Grid Directory

The recommended workflow uses `setup_grid.sh` to copy all necessary scripts into a dedicated working directory and configure the Condor submit files automatically:

```
# Run from the T-GRINS repository root
bash tools/setup_grid.sh Grid_SiFe_GAL

cd Grid_SiFe_GAL

# Copy your model parameter table
cp /path/to/tlusty-input.dat .

# Set the queue count to the number of models
wc -l tlusty-input.dat       # e.g. 626
# Edit tlusty_grid.sub: queue 626

# Submit
condor_submit tlusty_grid.sub
```

4.2 Running a Single Model

```
# Grid mode: row 3 of tlusty-input.dat
julia pipeline/run_model_fe.jl -g tlusty-input.dat -l 3

# Config mode: all parameters in a TOML file
julia pipeline/run_model_fe.jl -c config.toml

# Quick mode: specify Teff and logg on the command line
julia pipeline/run_model_fe.jl -t 35000 -G 4.0

# Legacy mode (backward compatible): positional line number
julia pipeline/run_model_fe.jl 3
```

4.3 Monitoring and Collecting Results

```
# Check which models converged
julia tools/check_grid.jl

# Collect all JLD2 files into JLD2/ directory
julia tools/collect_jld2.jl

# Plot a single model
julia plot/plot_sed.jl model_35000_400/mod_sp.jld2 --save
python plot/plot_sed.py model_35000_400/mod_sp.jld2 --save
```

A typical output from `check_grid.jl`:

```
Checking 626 models from tlusty-input.dat...
==============================================================
 GRID STATUS
==============================================================
```

```
  OK          : 553
  Missing     : 73
  Suspicious  : 0
  Total       : 626
================================================================
Written: rerun-input.dat (73 models to rerun)
```

5 Input File Format

5.1 Grid Mode Input

The grid input file is a plain-text table with one model per line. The third column (metallicity Z relative to solar) is optional; if absent, the value of `Z_SOLAR` defined at the top of `run_model_fe.jl` is used. Comment lines starting with `#` are ignored.

```
# Teff [K]  logg [cgs]  Z [optional, relative to solar]
35000.0     4.0
35000.0     4.0         0.5
50000.0     4.3         1.0
```

Metallicity hierarchy. The value of Z for a given model is determined in the following order of priority: (1) third column of the input file (grid mode only); (2) `Z` key in `[abundances]` or `[model]` in `config.toml` (single mode); (3) `Z_SOLAR` constant at the top of `run_model_fe.jl`.

5.2 Config Mode Input

In single-model mode, all parameters are read from a TOML file. The `config.toml` at the repository root serves as a documented template:

```
[model]
teff = 35000.0  # effective temperature [K]
logg = 4.0      # log surface gravity [cgs]

[abundances]
Z  = 1.0        # global metallicity (all metals Z > 2)
# fe = 2.0      # override Fe to 2x solar (other metals follow Z)
# he = 1.5      # He is never scaled by Z; set explicitly if needed

[tlusty]
nd     = 50
nlambd = 8
chmax  = 1.0e-5

[synspec]
sp_cutof = 0.01
# segments = [[50.0, -900.0, 1.0e-2], [900.0, -50000.0, 1.0e-4]]

[output]
step_ang  = 0.5
keep_ascii = false
```

All fields except `[model].teff` and `[model].logg` are optional. Unspecified parameters take the default values listed in Section 7.

6 Physical Setup

6.1 NLTE Pipeline Steps

The pipeline runs four TLUSTY calculations followed by one SYNSPEC call. Table 1 summarises the role of each step and the key parameters that differ between them.

Table 1: Summary of the five pipeline steps.

Step	Name	Treatment	Key parameters
1	`mod_lt`	LTE gray atmosphere	`NLAMBD=3, ISPODF=0`
2	`mod_nc`	NLTE continuum, H He CNO	`NLAMBD=6, ISPODF=0`
3	`mod_nc_fe`	NLTE + Si S Fe (LTE ODFs)	`NLAMBD=8, ISPODF=1`
4	`mod_nl`	NLTE full lines	`NLAMBD=8, ISPODF=1`
5	`mod_sp`	SYNSPEC detailed SED	multi-segment

The intermediate step `mod_nc_fe` is the key innovation of this pipeline. By introducing the Fe superline blanketing (`ISPODF=1`) at the continuum level before solving the full NLTE line problem, the temperature structure $T(\tau)$ can absorb the back-warming effect of the iron forest before the expensive NLTE level-population calculation. This greatly reduces the number of iterations needed in the final `mod_nl` step and significantly improves convergence for models with $\Gamma_e \gtrsim 0.3$.

6.2 Elements Included

Table 2: Atomic species included in T-GRINS models.

Treatment	Elements
Explicit NLTE	H, He, C, N, O, Si, S, Fe
EOS implicit (LTE)	Ne, Na, Mg, Al, P, Cl, Ar, K, Ca
Ignored	Li, Be, B, F, Sc, Ti, V, Cr, Mn

Fe ion stages explicitly treated:

- **B stars** ($T_{\rm eff} < 30\,000$ K): Fe I–III + bare Fe IV.
- **O stars** ($T_{\rm eff} \geq 30\,000$ K): Fe I–VI + bare Fe VII.

Fe superlines use Kurucz iron data (`gf260x.lin`, `gf260x.gam`) via `ISPODF=1` in the `mod_nc_fe` and `mod_nl` steps. The SYNSPEC line list `gfATO` covers all elements up to $Z \approx 92$ in LTE and contributes ∼1.48 million lines below 900 Å alone.

6.3 Eddington Parameter and Convergence Limits

The Eddington parameter for electron scattering is

$$\Gamma_e = \frac{\kappa_e L_\star}{4\pi c G M_\star} = \frac{\kappa_e \sigma T_{\rm eff}^4}{c\, 10^{\log g}}, \tag{1}$$

where $\kappa_e = 0.34$ cm^2 g^{-1} is the electron-scattering opacity for a solar-composition plasma. Models with $\Gamma_e \gtrsim 0.7$ are classified as near-Eddington and often fail to converge from a gray LTE starting point. Section 9 describes the retry strategy for these cases.

6.4 Metallicity Scaling

When $Z \neq 1$ (or when individual element overrides are specified), T-GRINS computes explicit logarithmic abundances relative to Asplund et al. (2009) solar values and writes them into the TLUSTY `.5` input files:

$$\log \frac{N_X}{N_H} = \log \left(\frac{N_X}{N_H}\right)_\odot + \log Z. \tag{2}$$

Hydrogen is always fixed at $N_H/N_H = 1$. Helium is not scaled by Z and must be set explicitly via the `he` key in `[abundances]` if a non-solar value is desired. When $Z = 1$ and no individual overrides are set, `abn = 0` is written and TLUSTY uses its internal solar defaults.

Recommended presets:

Z	Environment
1.0	Solar / Milky Way
0.5	LMC
0.2	SMC

6.5 SYNSPEC Wavelength Coverage

SYNSPEC is run in multiple segments to avoid an internal Fortran array overflow caused by the extreme line density in the far-UV ($\lambda < 900$ Å). Table 3 lists the default segments.

Table 3: Default SYNSPEC wavelength segments. Negative $\lambda_{\rm end}$ indicates vacuum wavelengths. `relop` is the opacity threshold relative to the local continuum: lines below this threshold are rejected.

Type	Seg.	$\lambda_{\rm start}$ (Å)	$\lambda_{\rm end}$ (Å)	`relop`
O star	1	50	900	10^{-2}
O star	2	900	50 000	10^{-4}
B star	1	500	50 000	10^{-4}

The two segments are concatenated in memory before interpolation onto the uniform output grid and saved as a single JLD2 file. Custom segments can be specified in `config.toml` (single mode) or by modifying the `segments` array in `run_model_fe.jl` (grid mode).

7 Configuration Parameters

7.1 TLUSTY Parameters

Parameter	Default	Description
`nd`	50	Number of depth points in the model atmosphere.
`nlambd`	8	Number of accelerated Λ-iterations (ALI) per global NLTE step. Higher values give more accurate NLTE populations but increase runtime.
`chmax`	10^{-5}	Convergence criterion: maximum relative change in any quantity between successive iterations.
`itek`	15	Global iteration number at which Kantorovich acceleration activates.
`iacc`	10	Global iteration number at which Ng acceleration activates. Setting both `itek` and `iacc` > 50 disables all accelerations (useful for near-Eddington models).
`ielcor`	-1	Electron temperature correction. -1 disables it, which is recommended for NLTE models.
`vtb`	auto	Microturbulent velocity [$\mathrm{km\,s^{-1}}$]. Default: 2 $\mathrm{km\,s^{-1}}$ for B stars, 10 $\mathrm{km\,s^{-1}}$ for O stars.
`taulas`	auto	Lower boundary optical depth. Default: $10^{2.5}$ for $\log g \geq 3.0$; 10^{2} for supergiants ($\log g < 3.0$).

7.2 SYNSPEC Parameters

Parameter	Default	Description
`segments`	auto	List of wavelength segments, each specified as `[start, end, relop]`. Negative `end` values select vacuum wavelengths. Defaults are given in Table 3.
`sp_step`	0.1	Internal SYNSPEC wavelength step [Å]. This value is written to `fort.55` but overridden by SYNSPEC's adaptive internal grid.
`sp_cutof`	0.01	Line wing cutoff: lines are dropped beyond this fraction of the Doppler width from line centre.

7.3 Output Parameters

Parameter	Default	Description
`step_ang`	0.5	Wavelength step of the uniform output grid [Å]. Increase to 1.0 for smaller files.
`keep_ascii`	`false`	If `true`, the raw SYNSPEC ASCII output files (`mod_sp.spec`, `mod_sp.cont`) are kept alongside the JLD2.

8 Output Format

8.1 JLD2 File Structure

Each converged model produces a compressed JLD2 file (`mod_sp.jld2`) stored in the model directory. JLD2 is an HDF5-based format and can be read with any HDF5-compatible library.

Table 7: Keys stored in each `mod_sp.jld2` file.

Key	Type	Units	Description
`lam`	Float32	Å	Wavelength grid (uniform step)
`hlam_spec`	Float32	$\mathrm{erg\,cm^{-2}\,s^{-1}\,Å^{-1}}$	Eddington flux with lines
`hlam_cont`	Float32	$\mathrm{erg\,cm^{-2}\,s^{-1}\,Å^{-1}}$	Continuum flux only
`teff`	Float32	K	Effective temperature
`logg`	Float32	cgs	Log surface gravity
`Z`	Float32	—	Metallicity ($Z/Z_\odot$)
`vtb`	Float32	$\mathrm{km\,s^{-1}}$	Microturbulent velocity
`star_type`	String	—	`"O-Star"` or `"B-Star"`
`step_ang`	Float32	Å	Wavelength step of output grid

The Eddington flux H_λ is related to the physical surface flux by $F_\lambda = 4\pi H_\lambda$. The flux at distance d from a star of radius $R_\star$ is

$$f_\lambda = F_\lambda \left(\frac{R_\star}{d}\right)^2 = 4\pi H_\lambda \left(\frac{R_\star}{d}\right)^2 . \tag{3}$$

8.2 Reading JLD2 in Python

```
import h5py
import numpy as np

with h5py.File("model_35000_400/mod_sp.jld2", "r") as f:
    lam       = f["lam"][:].astype(np.float64)   # Angstrom
    hlam_spec = f["hlam_spec"][:].astype(np.float64) # erg/cm^2/s/Ang
    hlam_cont = f["hlam_cont"][:].astype(np.float64)
    teff      = float(f["teff"][()])              # K
    logg      = float(f["logg"][()])
    Z         = float(f["Z"][()])                 # Z/Zsun
```

Listing 1: Reading a T-GRINS JLD2 file with h5py.

8.3 Reading JLD2 in Julia

```julia
using JLD2

d = load("model_35000_400/mod_sp.jld2")

lam      = d["lam"]       # Float32 array, Angstrom
hlam_spec = d["hlam_spec"] # Float32 array, erg/cm^2/s/Ang
teff     = d["teff"]      # Float32 scalar
logg     = d["logg"]
Z        = d["Z"]
```

Listing 2: Reading a T-GRINS JLD2 file with JLD2.jl.

8.4 Directory Contents After a Successful Run

```
model_35000_400/
|-- mod_lt.7        TLUSTY LTE model atmosphere (fort.7)
|-- mod_nc.7        NLTE continuum model
|-- mod_nc_fe.7     NLTE + Fe blanketing model
|-- mod_nl.7        Final converged NLTE atmosphere
|-- mod_nl.14       Low-resolution emergent flux (TLUSTY output)
|-- mod_sp.jld2     Compressed SED -- primary output
|-- mod_sp.iden     SYNSPEC line identifications (fort.12)
|-- mod_sp.log      SYNSPEC log
|-- param_*.nst     TLUSTY convergence parameters
|-- synspec.55      Last SYNSPEC input file
\-- *.5             TLUSTY atomic input files
```

9 Retry Strategy for Difficult Models

9.1 Motivation

Models with $\Gamma_e \gtrsim 0.5$ often fail to converge when started from a gray LTE atmosphere because the initial temperature structure is far from the NLTE solution. The TLUSTY iteration diverges typically at the NLTE continuum step (`mod_nc`), producing NaN values in the flux output.

The retry pipeline addresses this by using a nearby *converged* model as the starting point instead of the gray atmosphere. The neighbor model provides a realistic initial $T(\tau)$ profile, greatly reducing the number of iterations needed.

9.2 Generating the Retry List

```
julia tools/make_retry_list.jl
```

This reads `tlusty-input.dat` and the existing JLD2 files, identifies failed models, finds the best converged neighbor for each, and writes two output files:

- `retry-input.dat` — one line per failed model: `Teff logg`
- `retry-neighbors.dat` — one line per failed model: `Teff_neighbor logg_neighbor`

The neighbor selection metric penalises models with lower $\log g$ (less stable) and uses the Teff difference as a tiebreaker:

$$d = \sqrt{\Delta T^2 + \Delta g^2} + 10 \cdot \max(0,\, g_{\text{target}} - g_{\text{neighbor}}) + 0.1\,\Delta T, \tag{4}$$

where $\Delta T = |T_1 - T_2|/500$ and $\Delta g = |g_1 - g_2|/0.1$ are the normalised parameter differences.

9.3 Running the Retry

```
# After make_retry_list.jl, set queue count in retry_grid.sub:
#   queue <number of failed models>
condor_submit pipeline/retry_grid.sub

# Check results
julia tools/check_retry.jl
```

The retry pipeline runs three TLUSTY steps (nc → nc_fe → nl) starting from the neighbor's `mod_nl.7`. It uses more conservative convergence parameters than the main pipeline:

```
NLAMBD=3  # fewer lambda iterations (more stable for difficult models)
CHMAX=1e-3 # relaxed convergence criterion for nc step
ITEK=50   # disable Kantorovich acceleration
IACC=50   # disable Ng acceleration
```

Multiple retry rounds. It is normal and expected to run several rounds of retries. After each round, some newly converged models become available as neighbors for the remaining failed ones. The convergence rate typically improves in each successive round. In the SiFe GAL grid, five retry rounds were needed to reach the final 553/626 coverage.

10 HTCondor Integration

10.1 Submit File Structure

The Condor submit file `tlusty_grid.sub` looks like:

```
Universe  = vanilla
Executable = run_wrapper.sh
Arguments = $(Process)
Initialdir = /path/to/Grid_SiFe_GAL
Output    = Out/tlusty_$(Process).out
Error     = Error/tlusty_$(Process).err
Log       = condor.log
request_memory = 1024
request_cpus  = 1
queue 626
```

The `Arguments = $(Process)` passes the Condor process number (0-indexed) to `run_wrapper.sh`, which converts it to a 1-indexed line number and calls:

```
julia run_model_fe.jl -g tlusty-input.dat -l $((PROCESS + 1))
```

10.2 Memory Requirements

Typical memory usage per model is 400–600 MB including the Julia runtime and TLUSTY/SYNSPEC processes. Request at least 1 GB per slot:

```
request_memory = 1024   # MB
```

SYNSPEC memory with enlarged arrays. After recompiling SYNSPEC with `MLIN0 = 3 000 000`, the SYNSPEC process alone requires ∼500–800 MB at runtime. On cluster nodes with less than 1.5 GB of RAM per slot, SYNSPEC may be killed silently by the OS, producing a degenerate 235-point output. Check available memory per node with `condor_status`.

11 Plotting

T-GRINS provides two SED plotting scripts with identical functionality, one in Julia and one in Python.

```
# Julia
julia plot/plot_sed.jl model_35000_400/mod_sp.jld2    # interactive
julia plot/plot_sed.jl model_35000_400/mod_sp.jld2 --save # save PNG

# Python
python plot/plot_sed.py model_35000_400/mod_sp.jld2    # interactive
python plot/plot_sed.py model_35000_400/mod_sp.jld2 --save # save PNG
```

Both scripts display $\log \lambda$ vs $\log H_\lambda$ for the spectrum and continuum, mark standard reference lines (Lyman limit, Lyα, Hβ, Hα), and include the model parameters in the plot title. When `--save` is specified, the PNG is written to the same directory as the JLD2 file with the suffix `_sed.png`.

12 Troubleshooting

SYNSPEC produces only 235 points

This is caused by an internal Fortran array overflow in SYNSPEC. It occurs when the wavelength range starts below 900 Å and SYNSPEC is compiled with the default `MLIN0 = 1 200 000`. Recompile SYNSPEC with the enlarged arrays described in Section 2 and restart.

NaN in `mod_nc.13` from the first iteration

This indicates that the initial model (gray LTE or neighbor) is too far from the NLTE solution for the current $(T_{\rm eff}, \log g)$. The model directory is preserved with all intermediate files intact. Try the retry pipeline (Section 9) with a closer neighbor, or increase `ITEK` and `IACC` beyond 50 to disable all accelerations.

`mod_sp.spec` exists but contains only zeros or NaN

SYNSPEC can read the `mod_nl.7` structure file but the atmosphere is physically invalid (e.g., negative pressures in some layers). This happens when the retry pipeline produces a structure that converged numerically but not physically. The model cannot be rescued without a better starting point. In practice, this occurs only for models with $\Gamma_e \gtrsim 0.8$ and can be accepted as a fundamental convergence limit.

Julia package error: `TOML` not found

`TOML` is a standard library module in Julia 1.6+ and does not need to be installed separately. If the error persists, check the Julia version:

```
julia --version  # must be >= 1.6
```

Condor jobs finish immediately with exit code 1

Check the `Error/` directory for the corresponding `.err` file. Common causes are: (1) the environment variables `TLUSTY`, `LINELIST`, or `IRON` are not propagated to the Condor universe—add `getenv = True` to the submit file; (2) Julia is not in the `PATH` on the worker nodes.

13 Quick Reference

Command	Description
Setup	
`bash tools/setup_grid.sh <name>`	Create a configured grid directory.
Running models	
`julia pipeline/run_model_fe.jl -g file -l N`	Grid mode: run row N of input file.
`julia pipeline/run_model_fe.jl -c config.toml`	Single mode: run one model from TOML config.
`julia pipeline/run_model_fe.jl -t T -G g`	Quick mode: specify $T_{\rm eff}$ and $\log g$ directly.
`condor_submit pipeline/tlusty_grid.sub`	Submit full grid to HTCondor.
Monitoring and retry	
`julia tools/check_grid.jl`	Report OK / Missing / Suspicious counts.
`julia tools/make_retry_list.jl`	Generate retry pairs for failed models.
`condor_submit pipeline/retry_grid.sub`	Submit retry jobs to HTCondor.

Command	Description
`julia tools/check_retry.jl`	Report retry results.
Post-processing	
`julia tools/collect_jld2.jl`	Collect all JLD2 files into `JLD2/`.
`julia tools/make_jld2.jl`	Regenerate JLD2 from existing ASCII output.
`julia plot/plot_sed.jl file.jld2`	Plot SED (Julia).
`python plot/plot_sed.py file.jld2`	Plot SED (Python).

References

Asplund, M., Grevesse, N., Sauval, A. J., & Scott, P. 2009, *Annual Review of Astronomy and Astrophysics*, 47, 481

Hubeny, I., & Lanz, T. 2017a, arXiv:1706.01859

Hubeny, I., & Lanz, T. 2017b, arXiv:1706.01860

Hubeny, I., & Lanz, T. 2017c, arXiv:1706.01861

Hubeny, I., Allende Prieto, C., Osorio, Y., & Lanz, T. 2021, arXiv:2104.02829

Lanz, T., & Hubeny, I. 2003, *The Astrophysical Journal Supplement Series*, 146, 417

Lanz, T., & Hubeny, I. 2007, *The Astrophysical Journal Supplement Series*, 169, 83